WI1102 — FINAL PROJECT REPORT

Group Number: Group 06

Project Title: Smart Vending Machine

Source Code Link: WI1102_Project1.ipynb

Presentation Video Link: WI1102 • Group 06

Group Members & Task Allocation:

No.	Name	SID	Role	Responsibilities
1	Yustinus B. A. Nugroho	13526004	Backend	Created the whole vending machine system
2	M. H. Akemi Pasha	13526044	Writer and Slides	Create and tidy up the final report. Make the presentation slides
3	Marvell Arya P.	13526012	Writer	Final Report
4	Bagaskhara Satria P. D.	13526008	Writer and Slides	Final Report and Presentation Slides

Table 1, List of group members and their roles

General System Description

This project models a smart vending machine system by using Python. The primary objective is to demonstrate how core computational thinking concepts, specifically two-dimensional (2D) arrays, modular subprograms, iterative loops, and conditional logic can be integrated to manage product selection, payment processing, and real-time inventory tracking.

The system organizes four products within a 2 × 2 grid using three parallel 2D arrays to store product names, prices, and stock quantities. Users select items by inputting row and column coordinates. Before initiating a simulated QRIS payment procedure, the program validates coordinate boundaries and verifies stock availability. Upon payment confirmation, the system removes the selected item's inventory by one and outputs a dispensing confirmation.

System testing confirmed that the program accurately processes valid transactions, rejects coordinates that are not listed, and blocks purchases of items that are sold out. However, testing also revealed key limitations: non-integer inputs trigger errors, the payment module accepts arbitrary input without verifying transaction status, and inventory data resets upon program termination. Future iterations should incorporate exception handling, direct payment cancellation logic, and consistent file storage.

Functional Decomposition

Decomposition Breakdown:

- **Catalog Display Module (**DisplayGrid**):** Accepts items, prices, stock, rows, and cols as formal parameters. Executes nested loops to traverse and render a formatted visual grid to the terminal displaying item coordinates, names, prices, and available stock.

- **Coordinate & Boundary Validation Module:** Evaluates raw inputs row_in and col_in against bounds $0 \leq row_in < ROWS$ and $0 \leq col_in < COLS$. Rejects selections before array retrieval if out of bounds to prevent memory indexing errors.

- **Inventory Stock Guard Module:** Checks whether stock[row_in][col_in] > 0. If positive, decrements the element by 1 upon transaction clearance; if zero, terminates the transaction branch and outputs an error message.

- **Cashless QRIS Processor (**ProcessQRISPayment**):** Accepts item_name and price as parameters. Formats the checkout receipt, presents the payment interface to the customer, and awaits confirmation input.

- **Session Lifecycle Manager:** Maintains continuous system operation through an overarching while loop governed by a Boolean state flag (running), evaluating user input choice ('y' or 'Y') to start again or end the session.

System Abstraction

Elements Included:

- **Parallel 2D Grid:** Modern vending machines use a row-and-column layout. In our system, we implemented it the same way.

- **Object Coordinates Verification:** In real vending machines (specifically the ones with an interactive touch screen instead of the traditional button vending machines), the user must input the numbers that is available on the vending machine, if not then the system will ask to input another number until it is verified.

```
================== VENDING MACHINE ==================
Row 0 -> [0][0] Cold Water (Rp3000 | Stock: 3)  [0][1] Soda Can (Rp7000 | Stock: 2)
Row 1 -> [1][0] Potato Chips (Rp5000 | Stock: 1)  [1][1] Chocolate Bar (Rp4000 | Stock: 4)
=====================================================
```

Figure 1, an image of the parallel grid from the source code

- **Inventory State Monitoring:** In-memory integer counters mirror internal drop sensors that keep track of available product stock.

- **Digital Payment Flow:** Like most modern vending machines in Indonesia, the system will use QRIS as the default payment in which the coding simulates this exchange through text prompts and waiting the user to enter the keypress.

```
Press ENTER after scanning and completing payment in app... enter
Payment Verified Successfully!
```

Figure 2, an image of the “Press ENTER” part from the source code

Elements Excluded:

- **Hardware and Cooling:** We didn’t include this because it doesn’t affect transaction logic.

- **Mechanical Workflow:** Every work done on the process (like moving an item from the layout to the dispenser) is done through one abstraction of

```
print("\n>>> DISPENSING ITEM: " + selected_item + " <<<")
```

```
>>> DISPENSING ITEM: Cold Water <<<
Transaction complete!
```

Figure 3, an image of the output of the dispenser

Pattern Recognition

- **Matrix Traversal Pattern:** The catalog presentation executes a standard $O(R \times C)$ nested iteration pattern, sweeping through row indices r from 0 to rows-1 and column indices c from 0 to cols-1 to dynamically assemble grid rows:

$$\forall r \in [0, \text{rows} - 1], \quad \forall c \in [0, \text{cols} - 1] \implies \text{Print}(items[r][c], prices[r][c], stock[r][c])$$

```
print("\n================== VENDING MACHINE ==================")
for r in range(rows):
    row_str = "Row " + str(r) + " -> "
    for c in range(cols):
        row_str += "[" + str(r) + "][" + str(c) + "] " + items[r][c] + " (Rp" + str(prices[r][c]) + " | Stock: " + str(stock[r][c]) + ")  "
    print(row_str)
print("=====================================================\n")
return
```

Figures 4 and 5, the LaTeX and image of the grid from the source code.

- **Boundary Guard Pattern:** Runtime protection follows a compound Boolean evaluation pattern before any array read operation occurs:

$$\text{IsValid} \iff (0 \leq \text{row_in} < \text{ROWS}) \land (0 \leq \text{col_in} < \text{COLS})$$

- **Pre-condition Depletion Check Pattern:** Prevents illegal negative stock states by checking the available quantity before initiating payment:

$$\text{CanTransact} \iff stock[\text{row_in}][\text{col_in}] > 0$$

```
# check if coordinate is valid
if (row_in >= 0 and row_in < ROWS and col_in >= 0 and col_in < COLS):
    # check stock availability
    if (stock[row_in][col_in] > 0):
        selected_item = items[row_in][col_in]
        selected_price = prices[row_in][col_in]

        # simulated QRIS Payment
        ProcessQRISPayment(selected_item, selected_price)

        # deduct stock
        stock[row_in][col_in] = stock[row_in][col_in] - 1
```

Figures 6 and 7, the LaTeX and image of the stock checker from the source code.

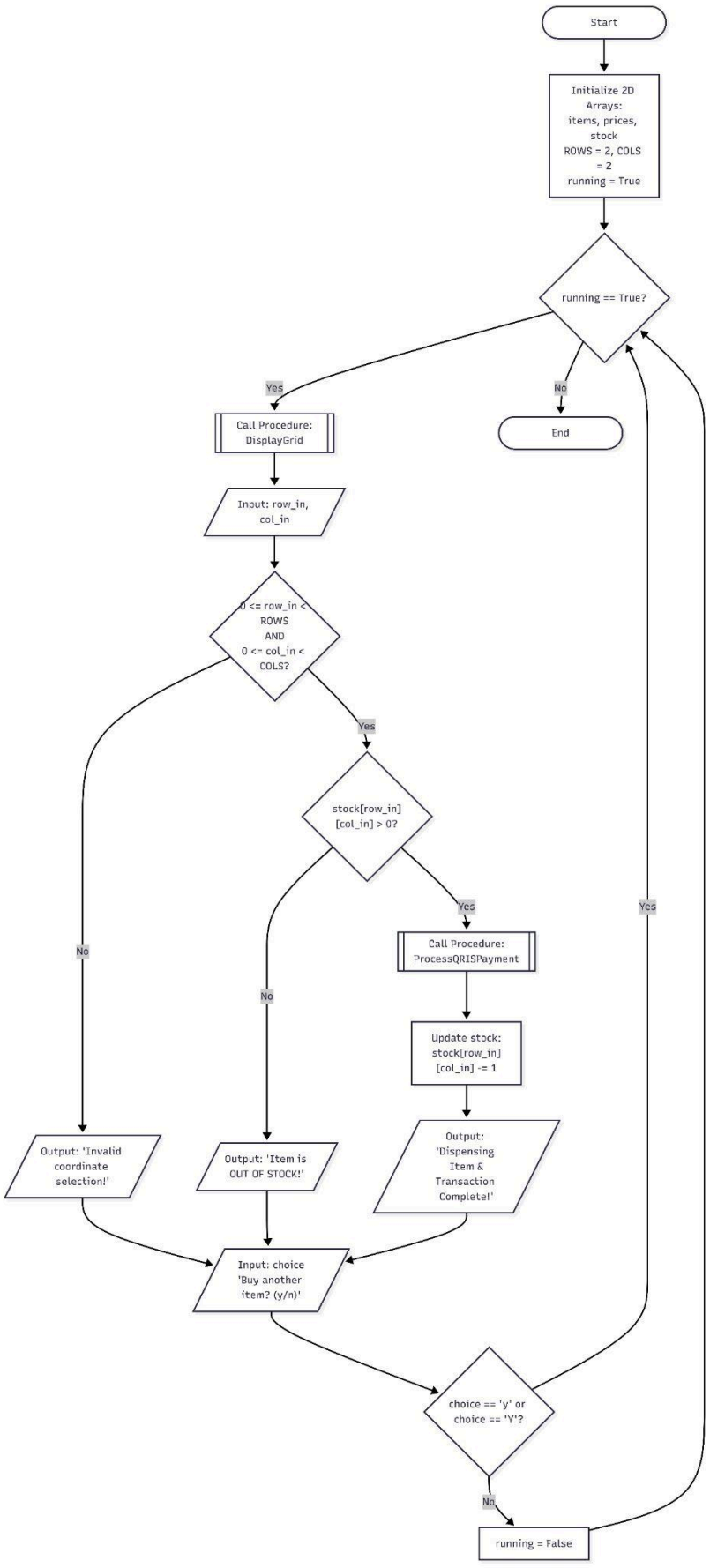
- **Sentinel-Controlled Loop Pattern:** Transaction continuity uses a sentinel flag pattern where a while loop remains active until user input explicitly deviates from

affirmative response characters (choice != 'y' and choice != 'Y').

```
choice = input("Would you like to buy another item? (y/n): ")
if (choice != 'y' and choice != 'Y'):
    running = False
```

Figure 8, image of the loop selection from the source code

Flowchart



Conclusion & Recommendations

The Smart Vending Machine project successfully illustrates how foundational programming constructs can model an automated retail system. By pairing parallel 2D arrays with modular subprograms, conditional guards, and while loops, the application reliably links product catalog data with real-time stock management. Functional testing verified that the program accurately decrements inventory after valid purchases, blocks out-of-range coordinate entries, and prevents transactions when stock is missing, while keeping payment processing and physical dispensing within a simulated scope.

To increase system reliability, future enhancements should implement try-except blocks to catch non-numeric inputs gracefully and prompt the user to re-enter valid coordinates without crashing the program. The payment subprogram should also be refined to evaluate specific confirmation or cancellation inputs, ensuring inventory is deducted only when a transaction is explicitly approved. Additionally, integrating external file handling (such as CSV or JSON storage) would allow inventory changes to persist across sessions. While full commercial deployment would require API integration with a live payment gateway, the final presentation and documentation should explicitly frame the current QRIS feature as a functional simulation.